93kbench User Manual

93kbench
February 17, 2011

Verigy Germany GmbH.
Herrenberger Strasse 130
71034 Boeblingen.

Contents

Chapter 1

93kbench user manual

1.1 Audience

The 93kbench User's Manual is intended for test and design engineers of Integrated Circuits 93000 SOC tester.

1.2 Terms used in this Manual

Term: Description:
ATE: *Automatic Test Equipment/tester*, Hardware used for quality check of each manufactured chip for compliance to the matching test program.

DUT: *Device Under Test* - The device model or the actual silicon.

IEEE: *Institute of Electrical and Electronics Engineers, Inc.*,continues to develop sophisticated yet complex modules to this standard language.

1.3 Introduction

The 93kbench is part of the closed loop design and test environment. The tool provides a link back from the production test program to the design environment. Using that link provides the designer a tester-like environment to verify the operation of the test program.

1.4 Concept

93kbench is designed to translate 93k test program files into a Verilog testbench.

1.4.1 Why should I use 93kbench ?

When the test engineer finally gets the silicon and starts de-bugging the test program, he needs to analyze failures. The test engineer in that stage needs to understand the cause of failures which could be:

- A design issue - The test is initially created by the design team. The designer might have used methodologies that are incompatible with the production test. For example, they might have used force to set a value to some internal DUT registers. Such operation is obviously not valid when running in production test environment. The designed, might not be aware of that.
- A conversion issue - The test engineer has converted the test from a design that is event driven to a test that is cycle driven.
  The conversion process might have been faulty. For example, the conversion might have neglected some signal in the test vectors. In such cases, the test will fail though the original test was accurate enough.
- A production issue - This is the real reason for testing - to find faults that are mismatch between the design model of the DUT and the device as produced in silicon.

So, the test engineer needs to analyze where each failure on the tester belongs (design, conversion or production). Using virtual tests provides a tool to analyze and clear the first two failure reasons and do so *before silicon* and without using the tester. Emulating the tester environment in design simulation reveals design and test compatibility issues as well as conversion issues. The test bench created by virtual test reflects the activity of the tester in the design environment.

1.5 Methodology

93kbench provides a complete flow from a test program for 93k ATE to the Verilog Test Bench.
The 93kbench reads the following elements:

1. 93K Test program directory location
2. 93k test flow file
3. 93k test suite name

The 93kbench creates a Verilog language test bench emulating the tester operation. The Verilog test bench is linked in the design environment with the DUT model and simulated together. Input signals are driven as in the test vectors according to the tester timing. Output signals are compared during each compare window and the DUT data is compared to the test vectors expected data. A failure occurs when the DUT data does not match the expected vector data.

1.5.1 Using testflow

This section describes running the tool using a testflow file. The testflow file contains test suites and links to timing, configuration and vector files. Every test suite has its own label.
Using a testflow file is simpler because all you have to do to activate it is to update the program main directory, provide the path to the testflow file and list the test suite names. All the information required for running the 93kbench is linked automatically.

EXAMPLE - Testflow file:

```
hp93000,testflow,0.1
language_revision = 1;

information
    device_revision = "1";
end

test_suites


testflow_demo:
   override = 1;
   override_tim_equ_set = 1;
   override_lev_equ_set = 1;
   override_tim_spec_set = 1;
   override_lev_spec_set =1;
   override_timset = 1;
   override_levset = 1;
   override_seqlbl = "WR_RD_Mode0";
   override_testf = tf_1;
   site_control = "parallel:";

end


context

context_config_file = "Demo.cha";
context_levels_file = "Demo.lvl";
context_timing_file = "Demo.tim";
context_vector_file = "Demo.pmfl";
context_attrib_file = "";
context_analog_control_file = "";
context_waveform_file = "";
context_routing_file = "";

end
```

1.6 Simulation results

The simulation results are in three forms:

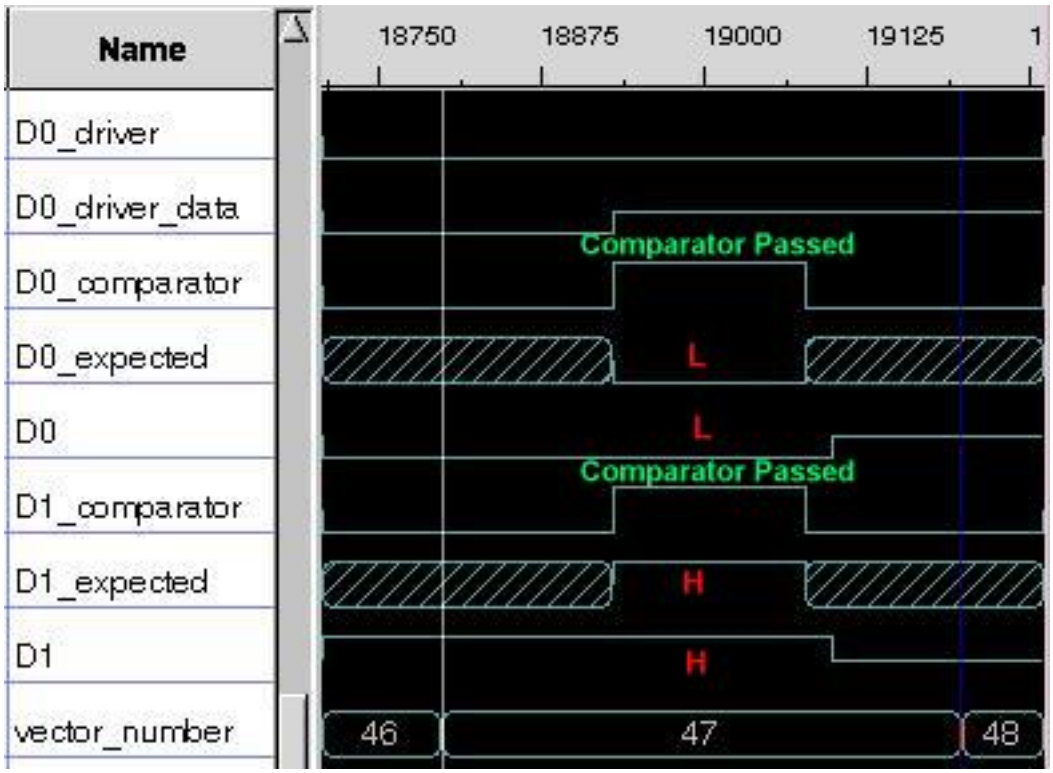
1. A log file during simulation reporting execution of each test vector, Fails and contentions. Contention is a situation that both the DUT and the tester model are driving the same signal.
   Log file example:

```
[ VTVECINF ][ vector information ][ pattern domain1_WR_RD_Mode0 ]
[ vector #0 ][ wft T0 ][ time 0 ]

[ VTCMPVAL ][ compare failed ][ D0 in domain1_WR_RD_Mode0 ]
[ vector #47 ][ wfc L][ real z ][ exp 0 ] [ time 19100000 ]

[ VTVECINF ][ vector information ][ pattern domain1_WR_RD_Mode0 ]
[ vector #47 ][ wft T6 ][ time 18800000 ]

[ VTDRVCNT ][ contention between device and tester ][ signal D0 ]
[ time 19040000 ]
```

2. A failure log reporting only failures.

```
[ VTCMPVAL ][ compare failed ][ D0 in domain1_WR_RD_Mode0 ]
[ vector #51 ][ wfc H][ real 0 ][ exp 1 ] [ time 20532000 ]
```

3. A VCD trace with indication of all DUT signals, the timesets used, indication of the compare window and drive state of each signal.



93kbench output files

The results from running 93kbench are simulation files and results per test. Creation of a Verilog test-bench emulating the behavior of the tester and considering timing pattern data. During run-time there is a creation of VCD trace of DUT signals and Tester activity. The test-bench is open for user customization. 93kbench builds *vt* directory under the project directory.The name of the directory is defined in the setup file(default =./vt). The *vt* directory contains the following files:

File Name	**Description**
*pattern_name*_pattern.bin	Binary pattern file, emulation of the tester pattern memory.
*pattern_name*_pattern_params.bin	Additional parameters to binary pattern file.
pattern_name.v	Test emulation module.
test_wrapper.v	Top level module, connecting the device top level to the tester module and defines the VCD signals.
*pattern_name*_timing.v	Tester timing file.
*pattern_name*_timing_control.v	Tester timing file.
board.hv	Board delay definitions, default:zero delay.
test.hv	Global definitions and timescale.
monitor.v	Defines control signals for each of the device signals.
tester.v	Tester engine main control.
vtrun1	Prototype for the Unix style shell script to compile and run the Verilog simulation, using explicit files.
vtrun2	Prototype for the Unix style shell script to compile and run the Verilog simulation, using verilog library option.

Under normal circumstances none of the above Verilog and binary files should be modified. The only exception is the top level (*test_wrapper.v*) file that defines tester to device connectivitya and delay definitions (*board.hv*) file. The *vtrun* script should be merged with a script running the normal device simulation.

1.7 Implementation

1.7.1 Linking to the device

The *vtrun* script should be merged with a script running the normal device simulation linked to the device.

Below is a *vtrun* file, after adding the link to the device model files marked:

test.hv \
board.hv \
$STILRWHOME/etc/verilog/vtw_wfgdef.hv \
$STILRWHOME/etc/verilog/vtw_stddef.hv \
$STILRWHOME/etc/verilog/vtw_logger.hv \
$STILRWHOME/etc/verilog/vtw_alldef.hv \
$STILRWHOME/etc/verilog/clockgen.v \
$STILRWHOME/etc/verilog/vtw_logger.v \
$STILRWHOME/etc/verilog/vtw_wfg.v \
WR_RD_Mode0_timing.v \
WR_RD_Mode0.v \
WR_RD_Mode0_timing_control.v \
test_wrapper.v \
tester.v \
monitor.v \
../Device_files/CNTREG.v \
../Device_files/DOWNCNTR.v \
../Device_files/I8253.v \
../Device_files/MODEREG.v \
../Device_files/OUTCTRL.v \
../Device_files/OUTLATCH.v \
../Device_files/READ.v

After editing the *vtrun* you need to:

1. Compile the DUT model with the Test Bench.
2. Run simulation.

1.7.2 The Results

After running simulation, the *vt* results directory will contain the following files:

File Name	**Description**
vtw.fail	Log of failures for the test. Empty if the test passes.
vtw.log	Simulation execution log file.
vtw.warn	Log of warnings.
pattern_name.vcd	A *VCD* formatted trace of the test, showing all signals of the device and indicators of failures, strobe windows and direction. Some additional useful information is present, including timeset, vector number, etc.

1.8 Invocation

The 93kbench can be controlled by command line parameters, setup file parameters or combination of both. The command line parameters have higher priority than setup file parameters, meaning the latter may be overridden using command line.

1.9 Using Command Line Interface (CLI) of 93kbench

1.9.1 Synopsys

93kbench [-help] [-version] [-timestamp] [-licwait] [-template] [-setup setup-file] [-work_dir <str>] [-precision <str>] [-dut_name <str>]

1.9.2 Command Line Options Description

The following are available command line options of 93kbench :

-help
: Display help message and exit.

-version
: Show version number and exit.

-timestamp
: Enable log timestamp.

-licwait
: Wait for the license, if not available. The application will wait indefinitely until a license becomes available.

-debug
: Generate intermediate file(s) to simplify debug (application dependent).

-template
: Generate setup template and exit. A file containing all supported setup options will be generated.

-setup
: Specifies path to setup file.

-work_dir
: Specifies output directory.

-precision
: simulation precision.

-dut_name
: device model name.

1.10 93kbench Setup file template content

Setup file template contains the following flags:

Verigy 93000 Configuration:

1. 93k test program main directory (Mandatory).
   _93k_testdir = ”

2. 93k test flow file (Mandatory).
   _93k_testflow = ”

3. 93k test suite name (Mandatory).
   _93k_testsuite = ”

   VTB Generator Configuration:

4. Name of the device model. (Optional)
   dut_name = ”dut”

5. Simulation precision. (Optional)
   precision = ”ps” possible values are ”ns”,”ps” and ”fs”

6. VT output directory. (Optional)
   work_dir = ”vt” place to put the VT files

7. Use of absolute paths in VT files, true by default. (Optional)
   use_absolute_paths = 1

You can get a setup file template by using the following command:

93kbench -template

You will get the following message :

```
The tool will generate template setup file in ’./vtgen_setup_template.py’ and silently exit.
```

In order to create a setup file, you can use the setup file template and edit the relevant flags.

In order to run the 93kbench tool, use the following command:
93kbench `-setup <Setup_Filename>`

1.11 93kbench run logfile example

```
Running Verigy 93000 VT (Version 1.1.1 [linux x86_64] 15-Dec-2009).
(c) Copyright 2000-2009 Verigy Ltd.

========================
= Verigy 93000 -> STIL =
========================

Reading Verigy 93000 Data Files from './93K\_prog'...
 ./93K_prog/testflow/testflow_demo.tf
 ./93K_prog/vectors/Demo.pmfl
 ./93K_prog/configuration/Demo.cha
 ./93K_prog/levels/Demo.lvl
 ./93K_prog/timing/Demo.tim
Done [reading verigy files]

Collecting Labels Info...
 ./93K_prog/vectors/zipped/WR_RD_Mode0.vecd
Done [labels info].

Analyzing Test Flow...
Done [analyzing flow]

Translating Labels...

Label 'WR_RD_Mode0':
Loading './93K_prog/vectors/zipped/WR_RD_Mode0.vecd'...
 Translating...
Status: OK

Done [Labels].

TestSuite 'testflow_demo':
 SPECSET   1 "AC1"
 WAVETBL     "wavetable 1"
 EQNSET    1 "equation set 1"
 TIMINGSET 1 "eq 1 ts 1"
 EQNSET    1 "Demo_set"
 LEVELSET  1 "Demo_set"
 SPECSET   1 "DC1"
 SQLB        "WR_RD_Mode0"
Status: OK

======= Translation Report =======
 testflow_demo ............. OK
==================================
 Total: 1  Passed: 1  Failed: 0

====================
=   STIL -> VTB    =
====================

Activating VTB Generator...
```

```
File: 'vt/stil/testflow_demo.stil'
 Pattern: WR_RD_Mode0
Done [vtb generator].
```